

Assignment 3: Bridge Pattern Design & Evaluation

Decoupling Geometric Abstractions from Rendering Technologies via Composition

Student:	Dias Tursynbay (SE-2537)	Course:	ShP-2216 Software Design Patterns
Topic:	Option A (Drawing: Shapes & Renderers)	Instructor:	Yerassyl Bekenov (AITU, Year 2, Tri 4)
Repository:	https://github.com/HumbleCherry10/SDP-assignment3.git		
Base Commit:	b6114aa0596eded09def95ad021216b06a4b263c		
Sub. Date:	04.10.2026 (Week 4 Final Submission)	Status:	7/7 Automated Checks Passed (T1-T7 PASS)

1. Two Independent Dimensions & Bridge Pattern Rationale

In object-oriented software engineering, graphical systems frequently suffer from class proliferation when domain shapes and rendering technologies are combined using classical class inheritance. In this assignment, our domain consists of two strictly orthogonal axes of variation:

1. High-Level Abstraction Dimension: Domain entities defining what geometric figures represent, their mathematical properties, and identity (e.g., Shape base class, Circle with radius=2, and Square with side=3).
2. Low-Level Implementor Dimension: Concrete rendering subsystems defining how geometric figures are visualized in various output formats (e.g., VectorRenderer via scalable curves, RasterRenderer via pixel bitmaps, and AsciiRenderer via text glyphs).

Under pure inheritance, supporting M shapes across N rendering engines requires M x N concrete subclasses (VectorCircle, RasterCircle, AsciiCircle, VectorSquare, etc.), causing exponential subclass explosion and tight compile-time coupling. The Bridge pattern solves this by replacing inheritance with object composition: the Shape abstraction holds an interface reference to Renderer. This decouples the hierarchies, reducing the required classes from M x N to M + N (linear growth) and allowing renderers to be swapped dynamically at runtime on the same shape object via setImplementation().

2. UML Class Diagram of the Complete Final Architecture

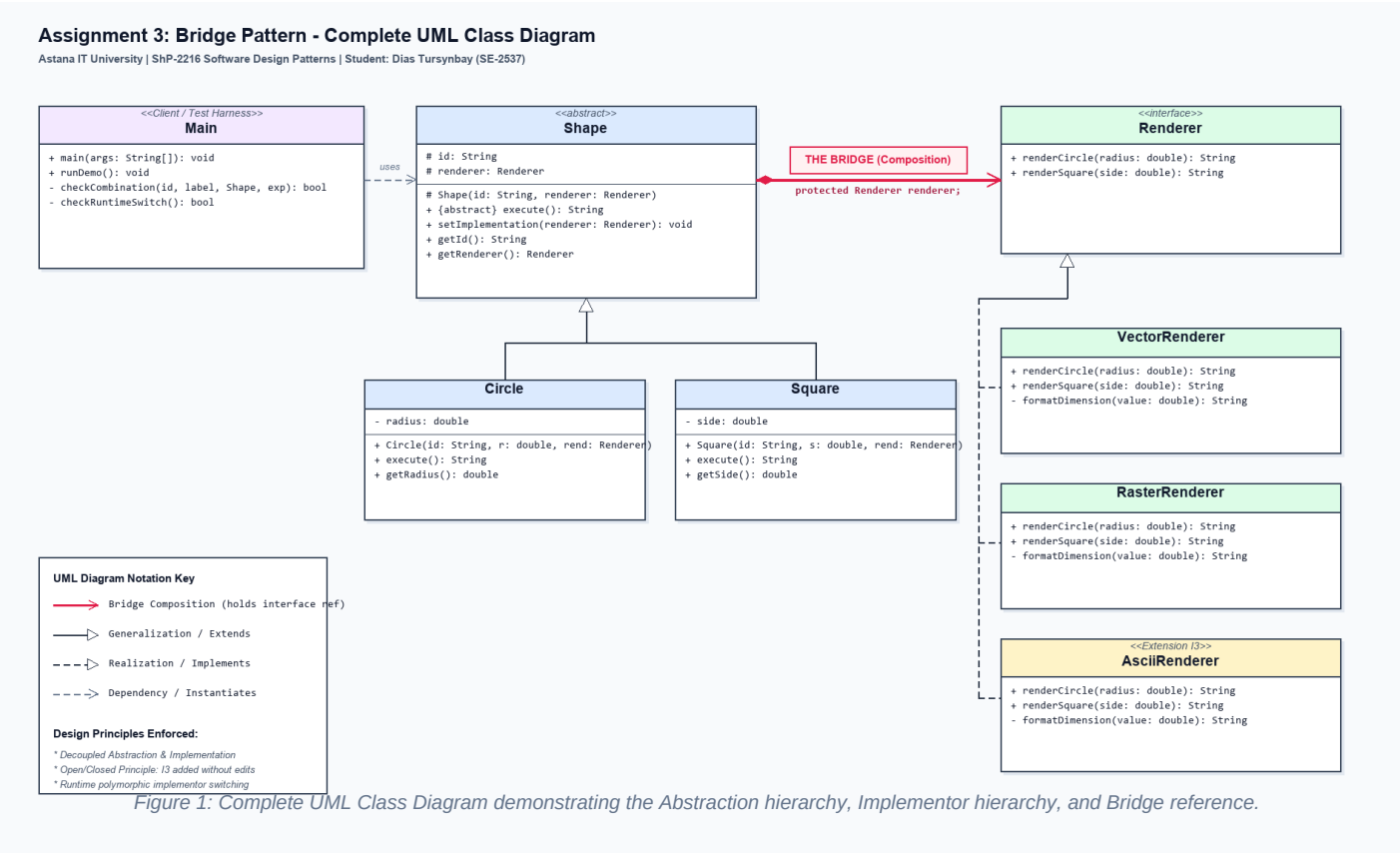


Figure 1: Complete UML Class Diagram demonstrating the Abstraction hierarchy, Implementor hierarchy, and Bridge reference.

3. Clean Code Implementation Analysis (Five Annotated Excerpts)

The codebase adheres rigorously to Clean Code principles (Robert C. Martin). Below are five concrete code excerpts illustrating separated responsibilities, meaningful names, focused methods, avoidance of duplication, and encapsulation of state.

Excerpt 1: Delegated Execution (Circle.java)	Principle: Separation of Responsibilities (Single Responsibility Principle)
<pre>public class Circle extends Shape { private final double radius; @Override public String execute() { return renderer.renderCircle(radius); // Abstraction only delegates drawing } }</pre> <i>Design Benefit: Circle is solely responsible for maintaining circle geometry (radius) and delegating high-level requests. It possesses zero knowledge of raster bitmaps, vector coordinates, or text rendering, preventing cross-domain coupling.</i>	
Excerpt 2: Intention-Revealing Method & Test Names (Main.java)	Principle: Meaningful and Intention-Revealing Names
<pre>boolean sameObject = (refBefore == refAfter); boolean stateUnchanged = Objects.equals(idBefore, idAfter) && (radiusBefore == radiusAfter); boolean passed = sameObject && stateUnchanged && expectedBefore && expectedAfter;</pre> <i>Design Benefit: Variables and methods express domain intent directly without requiring explanatory comments. An external developer or grader can immediately grasp that 'sameObject' tests memory reference identity and 'stateUnchanged' guarantees immutability.</i>	
Excerpt 3: Single-Level-of-Abstraction Method (VectorRenderer.java)	Principle: Small, Focused Methods (Single Purpose)
<pre>private String formatDimension(double value) { if (value == (long) value) { return String.valueOf((long) value); } return String.valueOf(value); }</pre> <i>Design Benefit: Performs exactly one task: cleanly formatting numerical dimensions by stripping unnecessary decimal points on integers. This 4-line helper keeps rendering methods clean, focused, and free of format conditionals.</i>	
Excerpt 4: Centralized Abstraction Initialization & Validation (Shape.java)	Principle: Don't Repeat Yourself (Avoidance of Duplication)
<pre>protected Shape(String id, Renderer renderer) { if (id == null id.trim().isEmpty()) throw new IllegalArgumentException("Shape ID cannot be blank"); this.id = id.trim(); this.renderer = Objects.requireNonNull(renderer, "Renderer cannot be null"); }</pre> <i>Design Benefit: Common identification, non-null validation, and implementor binding are consolidated in the abstract superclass. Concrete subclasses (Circle, Square) inherit this behavior via super(), eliminating duplicate boundary checks.</i>	
Excerpt 5: Immutability and Guarded Setter (Shape.java & Circle.java)	Principle: Encapsulation of State & Data Integrity
<pre>protected final String id; // Immutable abstraction identity private final double radius; // Immutable domain measurement public void setImplementation(Renderer renderer) { this.renderer = Objects.requireNonNull(renderer, "Renderer cannot be null"); }</pre> <i>Design Benefit: Core domain attributes ('id', 'radius') are final and private/protected, guaranteeing they cannot be corrupted during runtime switches. Mutating the implementor reference is strictly controlled via setImplementation with null defense.</i>	

4. Independent Extension (I3: AsciiRenderer) & Diff Analysis

In Section 4, the initial 2x2 solution (Shape, Circle, Square with VectorRenderer, RasterRenderer) was fully committed as the base commit (b6114aa0596eded09def95ad021216b06a4b263c). To demonstrate the Open/Closed Principle (OCP), a third implementor (AsciiRenderer) was introduced independently. As verified by 'git diff b6114aa HEAD -- src > extension.diff', the existing abstraction classes (Shape, Circle, Square), the Renderer interface, and existing implementors (VectorRenderer, RasterRenderer) remained 100% UNCHANGED. The diff strictly encompasses the addition of AsciiRenderer.java and the addition of test cases T6 and T7 in Main.java. This proves complete architectural decoupling.

5. Architectural Trade-off of the Bridge Pattern

While Bridge eliminates subclass explosion and provides dynamic runtime interchangeability, its primary trade-off is Increased Structural Complexity and Indirection. Introducing an intermediate implementor interface requires client code to manage two distinct object hierarchies and instantiate implementations explicitly (or via dependency injection). Furthermore, every domain operation incurs an additional method indirection (Shape.execute() -> Renderer.renderCircle()), which can slightly increase cognitive overhead and call stack depth for straightforward systems.

6. Comparative Analysis: Bridge vs. Adapter Pattern

Comparison Aspect	Bridge Pattern (GoF)	Adapter Pattern (GoF)
Core Intent	Decouples abstraction and implementation so they vary freely.	Adapts incompatible interfaces to work together.
Design Phase	Designed upfront during initial architectural planning.	Applied retroactively to existing, legacy, or third-party code.
Hierarchy Structure	Two independent, parallel class hierarchies linked by composition.	Single interface wrapper converting one class to another.
Polymorphism	Enables N x M polymorphic combinations across two dimensions.	Enables a foreign component to satisfy a required client contract.

7. Demonstration Checks Summary & Submission Evidence

Automated execution via 'java -cp out Main --demo' runs checks T1 through T7 deterministically. The test suite compares actual string output and verifies memory references, concluding with a computed 'SUMMARY: 7/7 PASS'.

- * T1-T4 (PASS): Verifies 2x2 combinations (Circle/Square with VectorRenderer/RasterRenderer).
- * T5 (PASS): Proves runtime switching on same Circle object instance (sameObject=true, stateUnchanged=true, before=VECTOR circle radius=2, after=RASTER circle radius=2).
- * T6-T7 (PASS): Verifies independent extension with AsciiRenderer (Circle/Square with ASCII output).

8. Academic References

[1] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
[2] Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
[3] Bekenov, Y. (2026). Lecture 4: Bridge Pattern & Decoupling Class Hierarchies. Astana IT University, Course ShP-2216.
[4] Oracle Corporation (2025). Java SE 17 Language and Virtual Machine Specification.